

# Notarized Anonymity Certificates Under Retries: Grinding-Resistant Evaluation via Transparency Logs, Public Randomness, and $\delta$ -Spending

Draft (2026-02-28; archive v125)

## Abstract

Empirical anonymity claims for anonymous overlays and DHTs are often published only after repeated evaluation, metric tuning, and parameter sweeps until “the numbers look good.” This stop-when-pass workflow can drive the probability of approving an unsafe build arbitrarily close to 1, even when each individual check has false-approval probability at most  $\delta$ . We give a compact release discipline robust to retries: (i) a summable  $\delta$ -spending schedule across attempts (and within-attempt metric libraries), (ii) notarized manifests that precommit code, parameters, and evaluation randomness, and (iii) publication of every initiated attempt to an append-only transparency log with beacon/VRF binding so aborts and selective disclosure are auditable. We prove that the disclosed budget upper-bounds false approvals under optional stopping and include toy simulations illustrating inflation under naive workflows and recovery under log-and-spend.

**Compilation note.** This paper is brief-by-design. A companion addendum collects extended figures, log-hardening discussion, and implementation templates (Eval 1A).[85]

**Scope.** We specify a *notarized* evaluation workflow for anonymity claims: how to bind a reported privacy budget to a concrete release, so that retries, audits, and downstream consumption remain checkable.

**Imports.** We import (i) schedule-only retry wrappers and downgrade-surface discipline (Operational A), (ii) auditable trace interfaces (Operational B; Synthesis 3), and (iii) ABOM/OINL claim binding (Anonymity C). We treat the beacon/VRF binding and transparency-log interface as owned by the primitives note (Synthesis 9).[19] Budget semantics are imported from odds inflation and the endpoint bridge (Anonymity A; Synthesis 5).[25, 26, 27] Notation conventions for keys, state, transcripts, and transcript projections follow the series notation sheet (Synthesis 8). Exposure-normal-form identity (ENF-ID) is imported from the trace-interface note (Synthesis 3), and threat/window declarations (TW-ID) from the threat-window note (Synthesis 4); receipt-lineage semantics are imported from the receipt-schema note.[21, 22, 23, 24]

**Exports.** Exports a minimal certificate bundle: an OINL label, a release receipt, and a short verifier report (including many-testing adjustments) that reproduces the stated anonymity bound under the declared retry policy.

**Artifact policy.** This archive is source-only; supporting artifacts (logs, traces, scripts, certificates, and verifier outputs) are available on request as a single bundle, committed by ABOM/OINL and governed by the series artifact policy (Synthesis 6).[20]

**Certificate object (informal).** Exports a grinding-resistant *notarized evaluation receipt*: public randomness, sample commitments, run digests, and an auditor-verifiable trail that pins down retries and many-testing adjustments.

## Contributions.

- We formalize how retries, parameter sweeps, and selective disclosure can turn “level- $\delta$ ” evaluations into near-certain acceptance, even when a single run is conditionally valid.
- We give a compact mitigation: a public *spending schedule* across logged attempts, coupled with pre-commitment to beacon/VRF-derived randomness, so that aborts and retries cannot inflate false releases beyond the disclosed budget.
- We provide a notarized publication checklist (manifest + transparency log + witness/gossip hooks) that lets readers verify the retry discipline without needing all artifacts inline.

## 1 Context in the series

This note is a *deployment wrapper* around prior technical work in the series: the privacy/anonymity objective is the same (e.g., TV leakage bounds, stopping-time leakage, and padding compilers), and we aim to publish short certificates rather than full artifacts. We therefore do *not* re-derive the anonymity objective or padding theory; see [28, 29, 30, 31]. Instead, we address the practical question: *how do we publish a certificate that remains valid when the evaluator can retry, sweep, and selectively disclose?* A companion note develops advantage contracts for stealth audits and audit-aware behavior; see [32].

**Series vocabulary (shared).** We use a small, consistent vocabulary across the series to keep papers non-redundant. A *plan id* names an evaluation definition (trace schema, attempt unit, within-attempt library/correction, and spending schedule); changing any evaluation knob implies a new plan id. An *epoch* (or window) is the declared validity interval of a plan (e.g., a wall-clock period or a beacon-derived id); changing epochs or restart conditions should be explicit. An *attempt* is one execution of a plan id over a predeclared epoch/window; retries, aborts, and detector changes are attempts and are logged. A *manifest* is a signed, hash-addressed statement committing code hash, plan id, parameters, and a randomness binding, and pointing to an offline artifact bundle (“available on request”).

## 2 Problem: retries turn “level- $\delta$ ” into “almost surely pass”

Consider an evaluation attempt that outputs a statistic and a binary decision *approve* / *reject*. If the build is unsafe (violates the claimed bound), assume the attempt approves with probability at most  $\delta$ . A common release gate repeats attempts until the first approval:

approve if  $\exists t \leq T$  such that attempt  $t$  approves.

Figure omitted in source-only release. Requested file:  
figures/within\_attempt\_metric\_library.png.

Figure 1: Within-attempt many-testing inflation: even with a summable spending schedule across attempts, sweeping many metrics/slices inside each attempt inflates false approvals unless the attempt budget is split (e.g., Bonferroni).

| Object                          | Practical budgeting rule                                                                                                                                                                                               |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Attempt index $t = 1, 2, \dots$ | Choose summable weights $w_t$ (e.g., $w_t \propto 1/t^2$ or $w_t \propto 2^{-t}$ ); set $\delta_t = \delta w_t$ .                                                                                                      |
| Within-attempt library          | Precommit $M =  \mathcal{M}  \cdot  \mathcal{S}  \cdot  \mathcal{R} $ (metrics $\times$ slices $\times$ seeds/restarts) and run each check at level $\delta_t/M$ (Bonferroni) or another precommitted correction [31]. |
| Aborts / retries                | Treat each abort/retry as consuming its attempt budget $\delta_t$ (and log it), so grinding is explicitly paid for.                                                                                                    |
| Publication                     | Publish only the first attempt whose full library passes; log the full manifest (including the library definition and all results).                                                                                    |

Table 1: A concrete  $\delta$ -budgeting recipe that blocks grinding across attempts and within-attempt sweeps with minimal operational overhead.

Under independence this approves an unsafe build with probability  $1 - (1 - \delta)^T$ , which can be close to 1 even for modest  $T$ . Worse, the evaluator can adaptively vary slices, metrics, or random seeds (“many testing”) [31].

**This is the security analogue of “p-hacking”.** The statistical failure mode here is not special to anonymity: it is the same “researcher degrees of freedom” problem as optional stopping and undisclosed analysis flexibility, popularized as *p-hacking* and the *garden of forking paths* [69, 70]. In our setting the degrees of freedom are operational (which traces, which slices, which parameters, when to stop), but the consequence is identical: nominal “level- $\delta$ ” gates can be driven toward near-certain approval unless retries and within-attempt sweeps are explicitly budgeted and logged.

**Many metrics per attempt are also retries.** Even if attempts are budgeted with a summable schedule, a common workflow is to run a *library* of metrics/slices per attempt and publish the most favorable result. Under an unsafe null, sweeping  $M$  metrics at level  $\alpha$  inflates the per-attempt false approval probability to  $1 - (1 - \alpha)^M$ . Figure 1 illustrates that attempt-level spending alone does not prevent this inflation; the per-attempt budget  $\delta_t$  must also be split across the within-attempt library (e.g., Bonferroni  $\delta_t/M$ , or another precommitted multiple-testing correction; see also [31]).

Figure 2 shows this inflation in a toy model. The companion addendum includes an analogous illustration for metric/slice/seed sweeping and other within-attempt grinding patterns. [85]

### 3 Threat model: how published anonymity certificates get gamed

We focus on *evaluation-aware* release workflows:

- **Stop-when-pass / rerun-until-pass.** Repeat the same check until a favorable draw.

Figure omitted in source-only release. Requested file: `figures/release_stop_when_pass.png`.

Figure 2: Naive stop-when-pass release gates inflate false safety approvals (unsafe releases).

- **Selective disclosure.** Run many checks; publish only the successful ones and hide failures.
- **Post-hoc commitment.** Commit to code/parameters after observing outcomes.
- **Abort-after-beacon grinding.** If the evaluation randomness is derived from a public beacon, abort runs that look unfavorable.

We assume the evaluator can keep private artifacts; we only require that *any published certificate* be backed by a verifiable manifest and log entries.

## 4 Anytime-valid spending for repeated attempts

Retries are a form of *optional stopping*: if the evaluator can keep rerunning until something “looks good,” nominal level- $\delta$  gates cease to mean much. The simplest fix is to precommit a *spending schedule*  $\{\delta_t\}_{t \geq 1}$  with  $\sum_t \delta_t \leq \delta_{\text{tot}}$ . This is the time-Bonferroni analogue of the *alpha-spending* approach in sequential analysis [71], and it matches the spirit of anytime-valid inference based on e-values/e-processes [72, 75, 73, 74]. The series also treats related “stop-time” pitfalls on the *privacy side* (adaptive deadline choices) in a companion addendum [17]. Concretely, attempt  $t$  outputs either (i) a p-value  $P_t$  (valid under the unsafe null) or (ii) an e-value  $E_t$  (with  $\mathbb{E}[E_t] \leq 1$  under the unsafe null).

### 4.1 Summable spending rule

Fix a sequence  $\{\delta_t\}_{t \geq 1}$  with  $\sum_t \delta_t \leq \delta_{\text{tot}}$ . Approve at the first  $t$  such that either  $P_t \leq \delta_t$  (p-value gate) or  $E_t \geq 1/\delta_t$  (e-value gate).

**Lemma 1** (Precommitted spending is safe under retries). *Under the unsafe null, if each  $P_t$  is a valid p-variable (or each  $E_t$  is a valid e-variable), then the spending gate approves with probability at most  $\delta_{\text{tot}}$ , even if the evaluator chooses a stopping time adaptively.*

*Proof sketch.* For p-variables,  $\mathbb{P}(P_t \leq \delta_t) \leq \delta_t$  and a union bound yields  $\mathbb{P}(\exists t : P_t \leq \delta_t) \leq \sum_t \delta_t$ . For e-variables, Markov’s inequality gives  $\mathbb{P}(E_t \geq 1/\delta_t) \leq \delta_t$  and the same bound applies. More refined proofs appear in the anytime-valid inference literature [74].  $\square$

**Lemma 2** (Spending with a within-attempt library). *Suppose attempt  $t$  evaluates a precommitted library of  $M_t$  metrics/slices and approves if any metric passes. If each metric test within attempt  $t$  uses threshold  $\delta_t/M_t$  (Bonferroni), then under the unsafe null the overall approval probability across all attempts is at most  $\sum_t \delta_t \leq \delta_{\text{tot}}$  (even under adaptive stopping).*

*Proof sketch.* Under the unsafe null, within attempt  $t$  the union bound gives  $\mathbb{P}(\text{attempt } t \text{ approves}) \leq \sum_{m \leq M_t} \delta_t/M_t = \delta_t$ . Applying Lemma 1 to the attempt-level events yields the result.  $\square$

### 4.2 Abort is spend

If an evaluator can *abort* after seeing information correlated with outcomes (e.g., a beacon-derived seed), aborts must count as spent attempts.

**Lemma 3** (Abort-after-beacon grinding must be budgeted). *If an evaluator can conditionally abort attempts after learning any partial information about the attempt outcome, then a spending proof requires counting each initiated attempt (including aborted ones) against the spending schedule.*

*Proof sketch.* The spending guarantee (Lemma 1) bounds  $\mathbb{P}(\exists t : \text{approve at } t)$  by  $\sum_t \delta_t$  only when each initiated attempt is accounted for. If aborts are free, the evaluator can implement rejection sampling: reveal a beacon-derived seed  $U \sim \text{Unif}[0, 1]$ , define a valid p-variable as  $P = U$ , and abort whenever  $U > \delta_1$ . Since aborted attempts do not consume budget, the evaluator can repeat until  $U \leq \delta_1$  occurs, which happens almost surely, forcing approval with probability 1 despite a nominal level- $\delta_1$  gate. Counting every initiated attempt (including aborted ones) against the schedule restores the union-bound argument, because each peek/abort consumes the corresponding  $\delta_t$ .  $\square$

The companion addendum includes a toy illustration of abort-after-beacon grinding (and partial-leakage grinding) and how log-and-spend blocks it. [85]

## 5 Notarized manifests and transparency logs

Spending controls retry inflation, but it does *not* prevent *selective disclosure*: an evaluator could run many attempts privately and publish only the favorable one. To make the retry budget auditable, we require that every *initiated* attempt be committed *before* outcomes are observed and later accounted for in an append-only public record.

We use a standard transparency-log pattern: each attempt produces a signed, hash-addressed *manifest* that is appended to an append-only Merkle log with inclusion and consistency proofs. This is the same commitment surface used in Certificate Transparency (CT) [9] and in supply-chain transparency logs such as Sigstore/Rekor [18]. For concreteness, Sigstore bundles can embed transparency-log entry material—including a Signed Entry Timestamp (SET, an inclusion promise), an inclusion proof plus checkpoint, and optionally an RFC3161 timestamp—which together support offline verification of log inclusion and signing time.[64, 84] The paper keeps artifacts off-net: the manifest carries only hashes and pointers; the full archive remains “available on request.”

### 5.1 Manifest

Each attempt is associated with a signed *manifest* containing:

- code hash (commit, container digest, or tarball hash),
- evaluation plan identifier (metric family / slice library / spending schedule),
- parameter set and environment fingerprint,
- traffic model identifier (key distribution, probe window/epoch, vantage selection),
- bootstrap / snapshot hash (routing-table snapshot, peer-set capture, or bootstrap list used to initialize the evaluation),
- adversary / threat model identifier (e.g., compromise fraction and capability assumptions) when the evaluation depends on it,
- randomness commitment: either a VRF output [16, 7] or a public beacon round (e.g., drand [5] or NIST beacon [6, 44]); for robustness, it is reasonable to hash-mix multiple independent beacons when available,

| Field                      | Meaning                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| Subject hash               | Code/build identifier (commit, container digest, or tarball hash)                                        |
| Predicate type             | e.g., <code>.../anonymity-certificate/v1</code>                                                          |
| Plan id                    | Metric/slice library + spending schedule identifier                                                      |
| Exposure normal-form id    | Stable identifier / schema hash for the transcript projection whose budget is being o                    |
| Threat-window id           | Stable identifier / schema hash for the repetition unit + window/horizon + univers                       |
| Traffic model id           | Key distribution + probe window/epoch + vantage selection (binds evaluation degr                         |
| Bootstrap/snapshot hash    | Hash of bootstrap list / routing snapshot / peer-set capture used to initialize the at                   |
| Threat model id (optional) | Identifier for any adversary assumptions that the evaluation depends on (capabilities                    |
| Claim lineage (optional)   | Predecessor claim id plus relation tag ( <b>same-enf</b> , <b>coarsening</b> , <b>new-interface</b> ) wh |
| Attempt index $t$          | Binds the threshold $\delta_t$                                                                           |
| Randomness commitment      | VRF output or beacon round (precommitted)                                                                |
| Artifact hash              | Hash of retrievable (offline) archive                                                                    |
| Log context                | Log URL/id and checkpoint reference                                                                      |

Table 2: Minimal notarized manifest fields (portable encoding).

- the attempt index  $t$  (which binds the spending threshold  $\delta_t$ ).

Artifacts can remain off-chain; the manifest points to a retrievable archive by hash.

**Portable encoding (optional).** To keep certificates machine-checkable across ecosystems, a manifest can be encoded as an *in-toto Statement* wrapped in a *DSSE* envelope, matching common transparency ecosystems (e.g., Sigstore bundles) [62, 61, 60, 64]. The protocol here only needs the *hash-addressed* statement; full artifacts remain “available on request.” Recent IETF work on Supply Chain Integrity, Transparency, and Trust (SCITT) is standardizing a general architecture for signed-statement transparency services; our manifests can be viewed as SCITT signed statements and the log evidence as receipt material, enabling cross-ecosystem tooling reuse [82].

**Distributing trust roots.** To reduce reliance on out-of-band configuration, publish the log’s URL(s), public key(s), witness policy, and rotation schedule using an update-security framework such as TUF [59].

## 5.2 Append-only log

Manifests are appended to an append-only log, yielding a publicly auditable record of all attempts. A standard implementation is a Merkle-tree transparency log with consistency proofs (as in Certificate Transparency) [9].

If the publication surface is simply “signed checksums plus proofs” (rather than an arbitrary statement API), minimalist transparency designs such as Sigsum are also suitable and come with monitor-oriented tooling and offline verification workflows [55, 56, 57].

In practice, modern deployments increasingly use tile-backed, read-optimized logs (the Static CT API) to make monitoring cheaper and more robust at scale (e.g., Sunlight and other tile-based logs built atop Tesseract) [10, 33, 36, 51]. Mirroring protocols make it easier to keep independent read-only copies of the log (and to serve tiles from CDNs/object storage), improving availability for monitors and referees. [46] For portability, the log inclusion and consistency evidence can be packaged as an offline-verifiable proof bundle (e.g., C2SP `tlog-proof` / `tlog-checkpoint` formats)

Figure omitted in source-only release. Requested file: `figures/release_transparency_log.png`.

Figure 3: Publishing notarized manifests to a transparency log binds retries, spending, and randomness.

---

**Algorithm 1** Log-enforced attempt

---

**Require:** spending thresholds  $\{\delta_t\}$ , attempt index  $t$ , beacon/VRF randomness source

- 1: create manifest  $M_t$  with code hash, plan id, exposure normal-form id, parameters, beacon round / VRF output, and  $t$
  - 2: append  $M_t$  to transparency log; obtain inclusion proof
  - 3: derive evaluation seed from (manifest, beacon/VRF) and run evaluation
  - 4: publish result (statistic + p/e-value) with log inclusion proof
- 

so referees can verify a certificate without contacting the log operator [48, 47]. Figure 3 shows the minimal data flow.

### 5.3 Protocol

**Deterministic seed derivation.** A concrete default is  $\text{seed}_t := H(H(M_t) \parallel \text{beacon}(r_t) \parallel t)$ , where  $H$  is a fixed hash (e.g., SHA-256),  $H(M_t)$  is the manifest hash, and  $\text{beacon}(r_t)$  is the public randomness value for the committed round  $r_t$  (or a VRF output). This makes the evaluation randomness *recomputable* and blocks post-hoc seed choice.

**Canonical hashing of structured manifests.** If  $M_t$  is a structured object (e.g., JSON), then “hashing the manifest” must specify a canonical serialization; otherwise, semantically identical manifests can hash differently across languages or tooling. Two safe defaults are: (i) hash the raw bytes of a signed envelope payload (e.g., DSSE), or (ii) use a standard JSON canonicalization scheme such as RFC 8785 (JCS) before hashing.[15]

**Gap-free attempt indices and commit-charged spending.** To make retries auditable, attempt indices should be gap-free within a plan id and epoch: the log (or a monitor) flags missing indices as aborted attempts. Operationally, charge the attempt budget at *commit time* (when the manifest is appended) rather than at completion time, and treat missing reveals/results as spent aborts (or an operational alarm), so abort-after-beacon grinding cannot concentrate budget on favorable instances.

**Proposition 1** (When a notarized evaluation certificate extends an older one). *Suppose a publisher wants to compare a newly notarized evaluation certificate against a prior one, or to present the new certificate as a continuation of the old claim. This is meaningful only if the new certificate declares either (i) the same plan id, exposure normal form id, repetition unit, and public conditioning as the old one, or (ii) an explicit coarsening of the old exposure normal form under the same plan-level repetition unit and public conditioning. Otherwise the correct action is to mint a new claim object (with a predecessor pointer), not to present the change as a budget delta inside one continuing certificate lineage.*

*Proof sketch.* The retry-spending guarantee says how to account for *repeated attempts of one declared experiment*. If the plan id or covered transcript projection changes, the experiment itself has changed, so the statement “the later certificate is better/worse than the earlier one” is no longer a comparison

---

**Algorithm 2** Notarized release gate (approve-once)

---

**Require:** total budget  $\delta_{\text{tot}}$ , summable schedule  $\{\delta_t\}$ , max attempts  $T_{\text{max}}$

```
1: for  $t = 1$  to  $T_{\text{max}}$  do
2:   run Algorithm 1 and obtain p-value  $P_t$  (or e-value  $E_t$ )
3:   if  $P_t \leq \delta_t$  or  $E_t \geq 1/\delta_t$  then
4:     publish certificate: manifest list, log proofs, and the accepted attempt index  $t$ 
5:     return
6:   end if
7: end for
8: publish failure certificate (optional): manifest list and log proofs
```

---

of like with like. Explicit coarsening is the one safe exception because it preserves the old statement up to data processing on the public view.  $\square$

#### 5.4 Selective disclosure: what the log buys you (brief)

Spending controls retry inflation, but does not by itself prevent an evaluator from *hiding* failed attempts. The point of logging is that, once every initiated attempt is appended *before* evaluation (Algorithm 1), the evaluator cannot later publish only the favorable outcome without leaving a public record of the failed (or aborted) attempts.

A referee-facing acceptance rule is therefore simple: accept a certificate only if it includes (i) log inclusion proofs for a gap-free prefix of attempt indices  $t = 1, \dots, t^*$  under the stated plan id, exposure normal-form id, and epoch, (ii) a globally consistent checkpoint for those inclusions (e.g., witnessed/cosigned), and (iii) evidence that attempt  $t^*$  is the *first* attempt in that prefix to cross its threshold. Under this rule, *selective disclosure inside the protocol* becomes auditable: missing indices indicate aborts (still spent), and omitted manifests cannot be hidden because they are already in the public log.

#### 5.5 Split-view resistance: witnesses, gossip, and cross-logging (brief)

If the log operator can equivocate (show different histories to different verifiers), then log inclusion proofs cease to be a global commitment. Standard defenses are (i) checkpoint gossip/monitoring, (ii) witnessed checkpoints (cosigned tree heads), and optionally (iii) cross-logging checkpoints into an independent log. These mechanisms are orthogonal to the retry-spending guarantee, but they are needed if third parties are to treat the log as authoritative; see [13, 8, 12, 14]. Recent standards work is also beginning to cite these witness/cosignature building blocks (e.g., Merkle Tree Certificates) [79].

Operationally, this is becoming easier: public witness networks and directories reduce bespoke bootstrap and make it realistic for referees to demand witnessed checkpoints (or equivalent gossip evidence) for published claims [67, 68, 54].

#### 5.6 Reader verification checklist (minimal, artifact-light)

A published certificate can stay *small* while still being referee-checkable. At minimum, publish (or link to) a compact bundle containing: the spending schedule identifier, the accepted attempt index  $t^*$ , the manifest(s) and log proofs for attempts  $1, \dots, t^*$ , the beacon/VRF evidence, and the accepted test statistic.

A referee (or reader) can then verify, mechanically:

| Degree of freedom     | What to commit (minimum)                                                                                                                                    |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Key distribution      | Uniform hashes vs prefix-targeted / adversarial keys; key batch size; any reserved ranges.                                                                  |
| Vantage selection     | How start nodes are sampled; IP/AS diversity policy; whether probes come from stable monitors vs churning clients.                                          |
| Lookup semantics      | Iterative vs recursive; parallelism $\alpha$ ; termination rule; timeout / retry rule; caching behavior.                                                    |
| Routing state         | Bootstrap list / snapshot hash; routing-table freshness policy; churn model (if simulated).                                                                 |
| Overlay parameters    | $k$ (bucket size), ID length, eviction policy, replication factor; any protocol extensions (e.g., BEP-5 details [2], libp2p/IPFS Kad-DHT specifics [3, 4]). |
| Measurement transport | UDP/TCP choices, pacing, connection reuse; any middlebox traversal; clock sync assumptions.                                                                 |

Table 3: Anonymous-DHT evaluation knobs that should be fixed *up front* (plan id / manifest) to avoid hidden degrees of freedom.

1. **Budget accounting.** Confirm the disclosed schedule satisfies  $\sum_t \delta_t \leq \delta_{\text{tot}}$  and that each initiated attempt consumes its  $\delta_t$  (including aborts); confirm any within-attempt library uses a precommitted correction (e.g., Bonferroni  $\delta_t/M_t$ ).
2. **Precommitment + claim surface.** For each attempt  $t \leq t^*$ , verify that the signed manifest (code hash, plan id, exposure normal-form id, parameters, traffic model, snapshot hash) was *logged* before outcomes, using the inclusion proof and an append-only checkpoint/consistency proof.
3. **Global log view.** Verify the checkpoint is globally consistent (e.g., via a witnessed/cosigned checkpoint and published witness policy, or via monitor/gossip evidence) [49, 50].
4. **Randomness binding.** Verify the evaluation seed is deterministically derived from the manifest and the stated beacon round or VRF output, so the evaluator could not grind by post-hoc seed choice.
5. **Gate correctness.** Verify the published  $P_{t^*}$  (or  $E_{t^*}$ ) crosses the disclosed threshold  $\delta_{t^*}$  (or  $1/\delta_{t^*}$ ), and that the certificate is indeed the *first* passing attempt in the log.
6. **Lineage discipline.** If the certificate claims continuity with an earlier one, verify the declared predecessor and relation tag are compatible with Proposition 1; otherwise treat the output as a new claim object rather than a budget update.

This is enough to make the retry discipline auditable *without* publishing full artifacts inline; hashes and “available on request” archives suffice.

## 6 Practical checklist for publishing anonymity certificates

Table 3 lists common “degrees of freedom that silently change trace distributions in anonymous-DHT evaluations; for a non-gameable certificate, bind them into the plan id / manifest rather than leaving them as undocumented evaluator choices.

**Example: protocol semantics are part of the plan.** In practice, “Kademlia-style” DHTs vary in which messages propagate and who answers lookups. For example, I2P’s network database uses a floodfill subset for netDb queries: store messages are flooded among floodfills using a Kademlia XOR metric, but leaseSet lookup queries are answered by the receiving floodfill without forwarding (to avoid a security issue). This changes both the set of in-path observers and the detectability surface for audits, so such forwarding/answering rules belong in the committed attempt definition.[80]

A certificate intended to be non-gameable should include:

1. Manifest schema, including code hash, parameters, the evaluation plan id, and the exposure normal-form id of the transcript projection being certified.
2. An explicit *attempt unit* definition for your anonymous DHT evaluation: the number of lookups per attempt, how keys are sampled (uniform, prefix-targeted, etc.), how start nodes/vantage points are sampled, and the epoch/window over which traces are collected. Bind this definition into the plan id and/or manifest (e.g., for Kademlia-family lookups [1]).
3. A fixed spending schedule  $\{\delta_t\}$  and an explicit statement of  $\delta_{\text{tot}}$ . A convenient default is  $\delta_t \propto 1/t^2$  (summable with a heavy head), or a geometric schedule when early approval is expected. If each attempt sweeps a library of metrics/slices, split  $\delta_t$  across that library (e.g., Bonferroni  $\delta_t/M_t$ ) as in Lemma 2.
4. A public log inclusion proof for every initiated attempt (including aborted ones).
5. A randomness binding: VRF output [16, 7] or a public beacon [5, 6] fixed before outcomes are observed.
6. A short “artifacts on request” statement keyed by archive hashes.

## 7 Takeaways

- **Retries are a release attack:** repeated evaluation and selective disclosure can inflate false approvals to near certainty even if each run is “level- $\delta$ ”.
- **Log-and-spend is the minimal fix:** bind every attempt (including aborts) into an append-only log, and publish a summable  $\delta$ -spending schedule (plus within-attempt splits) so optional stopping cannot exceed the disclosed budget.
- **Bind randomness and degrees of freedom:** commit probe selection via VRF/beacon and treat evaluation semantics (keys, vantages, retries, timeouts, caching) as plan-id parameters.
- **Make verification cheap:** a short manifest + inclusion proof lets third parties audit the retry discipline without shipping full artifacts inline.

## 8 Limitations and open problems

**Dependence and drift.** Spending controls Type-I error under optional stopping, but the underlying evaluation statistic must remain valid in the presence of dependence and distribution shift. In anonymous DHTs, dependence arises from repeated peers, caching, and diurnal latency. A conservative operational answer is to treat major configuration changes as new experiments (restart the spending schedule), and/or to run block-wise tests with a mixing assumption; tighter integration with overlay-specific dependence models is left for future work.

**Log equivocation.** If an operator can maintain multiple inconsistent logs, transparency loses force. Witnessing and gossip reduce the feasibility of split-view attacks but do not remove the need for diverse operational assumptions (independent witnesses/monitors, failure modes, and on-path adversaries). When high assurance is required, require witness cosigning (or cross-logging checkpoints to independent logs) and treat witness-set changes as configuration changes that restart certification.

## References

- [1] Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the XOR metric. In *IPTPS*, 2002.
- [2] BitTorrent Enhancement Proposal 5. DHT Protocol (Mainline). [https://www.bittorrent.org/beps/bep\\_0005.html](https://www.bittorrent.org/beps/bep_0005.html).
- [3] libp2p. Kademlia DHT specification. <https://github.com/libp2p/specs/blob/master/kad-dht/README.md>.
- [4] IPFS. IPFS Kademlia DHT specification. <https://specs.ipfs.tech/routing/kad-dht/>.
- [5] drand. Protocol specification. <https://docs.drand.lol/docs/specification/>.
- [6] NIST. Interoperable Randomness Beacons / NIST Randomness Beacon. <https://csrc.nist.gov/projects/interoperable-randomness-beacons>.
- [7] Silvio Micali, Michael Rabin, and Salil Vadhan. Verifiable Random Functions. In *FOCS*, 1999.
- [8] Ewa Syta et al. Keeping Authorities “Honest or Bust” with Decentralized Witness Cosigning. In *IEEE S&P*, 2016.
- [9] Certificate Transparency v2. RFC 9162. <https://www.rfc-editor.org/rfc/rfc9162.html>.
- [10] C2SP. Static Certificate Transparency API. <https://github.com/C2SP/C2SP/blob/main/static-ct-api.md>.
- [11] Chris Hartwig and Vincent Van Doordrecht. A next-generation Certificate Transparency log built on Cloudflare Workers. Cloudflare blog, 2025. <https://blog.cloudflare.com/azul-certificate-transparency-log/>.
- [12] Alexandra Dirksen et al. LogPicker: Strengthening Certificate Transparency Against Covert Adversaries. *PoPETs*, 2021. <https://petsymposium.org/popets/2021/popets-2021-0066.pdf>.
- [13] Rasmus Dahlberg et al. Aggregation-Based Certificate Transparency Gossip. *arXiv:1806.08817*, 2018.
- [14] Alexander Hicks. SoK: Log Based Transparency Enhancing Technologies. *arXiv:2305.01378*, 2023.
- [15] JSON Canonicalization Scheme. RFC 8785. <https://www.rfc-editor.org/rfc/rfc8785>.
- [16] Verifiable Random Functions (VRFs). RFC 9381. <https://www.rfc-editor.org/rfc/rfc9381.html>.
- [17] Anonymous. Stop-Time Padding Addendum (unique-only): Max-Mixture Separations and Tail-Sign Deadline Normal Forms. Series note (published), 2026. Wiki: [\[\[2026.01.26 - Mathematics: Stop-Time Padding Addendum, Max-Mixture Separations and Tail-Sign Deadline Normal Forms\]\]](#).
- [18] Zachary Newman et al. Sigstore: Software Signing for Everybody. In *CCS*, 2022.

- [19] Anonymous. Transparency Primitives for Proof-Carrying Anonymity Claims: Beacons, VRFs, and Append-Only Logs. Series synthesis note (Synthesis 9), The-One-True-Archive (2026).
- [20] Anonymous. Artifact and Disclosure Policy for the One-True-Archive (Source-Only Public Release). Series synthesis note (Synthesis 6), 2026. (This archive.)
- [21] Anonymous. Notation and Minimal Model for the One-True-Archive. Series synthesis note (Synthesis 8), 2026. (This archive.)
- [22] Anonymous. Trace interfaces for anonymous DHTs. Synthesis 3, 2026. (This archive.)
- [23] Anonymous. Threat models and repetition windows for anonymous DHTs. Synthesis 4, 2026. (This archive.)
- [24] Anonymous. Receipt line items: a minimal tuple schema for privacy claims. Synthesis 12, 2026. (This archive.)
- [25] Anonymous. Odds Inflation, Privacy Loss, and Endpoint Anonymity Bounds. Anonymity series Paper A, 2026. (This archive.)
- [26] Anonymous. Endpoint Metrics Bridge: Odds Inflation, PML/MaxL, and Identification Sizing. Series synthesis note (Synthesis 5), 2026. (This archive.)
- [27] Anonymous. ABOM and OINL for Anonymity Claims. Anonymity series Paper C, 2026. (This archive.)
- [28] Anonymous. *Scheduling and Compiler Techniques for Metadata-Hiding Overlay Lookups*. Series paper (published), 2026. Wiki: `[[2026.01.22 - Mathematics: Scheduling and Compiler Techniques for Metadata-Hiding Overlay Lookups]]`.
- [29] Anonymous. *Spectral Anonymity and Optimal Distinguishing Bounds for Random-Walk Delegation in (Possibly Directed) Overlays*. Series paper (published), 2026. Wiki: `[[2026.01.23 - Mathematics: Spectral Anonymity and Optimal Distinguishing Bounds for Random-Walk Delegation in (Possibly Directed) Overlays]]`.
- [30] Anonymous. *Optimal Poset-Feasible Padding: Knapsack, Transport, and Dual Certificates for TV Privacy*. Series paper (published), 2026. Wiki: `[[2026.01.29 - Mathematics: Optimal Poset-Feasible Padding: Knapsack, Transport, and Dual Certificates for TV Privacy]]`.
- [31] Anonymous. *Many-Testing Lower Bounds for Low-Latency Privacy on Hierarchical Overlays*. Series paper (published), 2026.
- [32] Advantage Contracts for Anonymous DHT Evaluations: Stealth Audits, Leakage Discovery at Scale, and Practical Padding Filters. Online manuscript in the author’s Anonymity / Anonymous DHT series, 2026.
- [33] Let’s Encrypt. Introducing Sunlight: a Certificate Transparency log implementation built for the Static CT API. Let’s Encrypt Blog, 2024. Accessed 2026-02 <https://letsencrypt.org/2024/03/14/introducing-sunlight>
- [34] Let’s Encrypt. Reflections on a Year of Sunlight. Let’s Encrypt Blog, 2025. Accessed 2026-02 <https://letsencrypt.org/2025/06/11/reflections-on-a-year-of-sunlight>
- [35] The Sunlight CT Log (Static CT API). sunlight.dev, 2026. Accessed 2026-02 <https://sunlight.dev/>
- [36] Static Certificate Transparency API. C2SP specification, 2026. Spec for tile/static CT endpoints (also mirrored at c2sp.org); accessed 2026-02 <https://c2sp.org/static-ct-api>
- [37] Easterday, Adit. I Built a New Certificate Transparency Log in 2024 – Here’s What I Learned. transparency.dev blog, 2025. Accessed 2026-02 <https://blog.transparency.dev/i-built-a-new-certificate-transparency-log-in-2024-heres-what-i-learned>

- [38] What Chrome is thinking about Certificate Transparency in 2024 and beyond. Chromium CT Policy mailing list, 2024. Accessed 2026-02 <https://groups.google.com/a/chromium.org/g/ct-policy/c/W7OS03SbrFo>
- [39] Certificate Transparency Log Policy. Chrome Certificate Transparency documentation, 2026. States acceptance of RFC 6962 logs and logs conforming to the static-ct-api C2SP specification; accessed 2026-02 [https://googlechrome.github.io/CertificateTransparency/log\\_policy.html](https://googlechrome.github.io/CertificateTransparency/log_policy.html)
- [40] Apple’s Certificate Transparency log program. Apple Support, 2026. Mentions static-ct-api conformance for logs; accessed 2026-02 <https://support.apple.com/en-us/103703>
- [41] Rekor v2 GA – Cheaper to run, simpler to maintain. Sigstore Blog, 2025. Accessed 2026-02 <https://blog.sigstore.dev/rekor-v2-ga/>
- [42] OpenSSF. Announcing the Sigstore Transparency Log Research Dataset. OpenSSF Blog, 2025. Accessed 2026-02 <https://openssf.org/blog/2025/10/15/announcing-the-sigstore-transparency-log-research-dataset/>
- [43] Trail of Bits. Catching malicious package releases using a transparency log. Trail of Bits Blog, 2025. Accessed 2026-02 <https://blog.trailofbits.com/2025/12/12/catching-malicious-package-releases-using-a-transparency-log/>
- [44] Kelsey, John. NIST IR 8213: A Reference for Randomness Beacons (Format and Protocol Version 2). NIST CSRC, 2019. Initial public draft published May 2019; accessed 2026-02 <https://csrc.nist.gov/pubs/ir/8213/ipd>
- [45] Transparency Log Tiles API (tlog-tiles). C2SP specification, 2026. Accessed 2026-02 <https://c2sp.org/tlog-tiles>
- [46] Transparency Log Mirrors (tlog-mirror). C2SP specification, 2026. HTTP protocol for mirroring transparency logs; accessed 2026-02 <https://c2sp.org/tlog-mirror>
- [47] Transparency Log Checkpoint Format (tlog-checkpoint). C2SP specification, 2026. Accessed 2026-02 <https://c2sp.org/tlog-checkpoint>
- [48] Transparency Log Offline Proof Format (tlog-proof). C2SP specification, 2026. Accessed 2026-02 <https://c2sp.org/tlog-proof>
- [49] Transparency Log Witness Protocol (tlog-witness). C2SP specification, 2026. Accessed 2026-02 <https://c2sp.org/tlog-witness>
- [50] Transparency Log Cosignature Format (tlog-cosignature). C2SP specification, 2026. Accessed 2026-02 <https://c2sp.org/tlog-cosignature>
- [51] Tessera: A Go library for tile-based transparency logs. transparency-dev GitHub repository, 2026. Accessed 2026-02 <https://github.com/transparency-dev/tessera>
- [52] Introducing TesseraCT. blog.transparency.dev, 2025. Announcement of TesseraCT (Static CT API implementation) built atop tile-backed transparency logs; accessed 2026-02 <https://blog.transparency.dev/introducing-tesseract>
- [53] Trillian: General Transparency (project site). google.github.io/trillian, 2026. Notes that Trillian is in maintenance mode and recommends tile-backed logs via Tessera; accessed 2026-02 <https://google.github.io/trillian/>
- [54] Valsorda, Filippo. Building a Transparent Keyserver. words.filippo.io, 2025. Practical integration of tile-backed tlogs; accessed 2026-02 <https://words.filippo.io/keyserver-tlog/>

- [55] Sigsum: Key-usage transparency with signed checksums. sigsum.org, 2026. Minimalist transparency log ecosystem; accessed 2026-02 <https://www.sigsum.org/>
- [56] Sigsum design document. sigsum project documentation, 2026. Design overview, including log, witness, and monitor roles; accessed 2026-02 <https://git.sigsum.org/sigsum/tree/doc/design.md>
- [57] Sigsum: Getting started (offline verification demo). sigsum.org documentation, 2026. Step-by-step workflow: log a checksum and verify offline; accessed 2026-02 <https://www.sigsum.org/getting-started/>
- [58] Tesseract: An implementation of the Static CT API based on Tessera. transparency-dev GitHub repository, 2026. Accessed 2026-02 <https://github.com/transparency-dev/tesseract>
- [59] The Update Framework (TUF) Specification (Latest). theupdateframework.github.io, 2026. Accessed 2026-02 <https://theupdateframework.github.io/specification/latest/>
- [60] DSSE: Dead Simple Signing Envelope (Specification). secure-systems-lab GitHub repository, 2026. Accessed 2026-02 <https://github.com/secure-systems-lab/dsse>
- [61] in-toto Attestation Framework v1: Envelope layer (DSSE). in-toto/attestation specification, 2026. Accessed 2026-02 <https://github.com/in-toto/attestation/blob/main/spec/v1/envelope.md>
- [62] in-toto Attestation Framework v1: Statement layer. in-toto/attestation specification, 2026. Accessed 2026-02 <https://github.com/in-toto/attestation/blob/main/spec/v1/statement.md>
- [63] SLSA Provenance v1.0. slsa.dev specification, 2026. Accessed 2026-02 <https://slsa.dev/spec/v1.0/provenance>
- [64] Sigstore Bundle Format. sigstore.dev documentation, 2025. Accessed 2026-02 <https://docs.sigstore.dev/about/bundle/>
- [65] Introducing Trillian Tessera: Tile-based Transparency Logs. blog.transparency.dev, 2024. Accessed 2026-02 <https://blog.transparency.dev/introducing-trillian-tessera>
- [66] Tessera v1.0 is here. blog.transparency.dev, 2025. Accessed 2026-02 <https://blog.transparency.dev/tessera-v1-0-is-here>
- [67] Can I Get a Witness (Network)?. blog.transparency.dev, 2024. Overview of public witness networks; accessed 2026-02 <https://blog.transparency.dev/can-i-get-a-witness-network>
- [68] TrustFabric Witnesses (conforming to C2SP tlog-witness). transparency.dev, 2025. List of public/dev witnesses; accessed 2026-02 <https://transparency.dev/witnesses/>
- [69] Simmons, Joseph P. and Nelson, Leif D. and Simonsohn, Uri. False-Positive Psychology: Undisclosed Flexibility in Data Collection and Analysis Allows Presenting Anything as Significant. *Psychological Science*, 22(11), 1359–1366, 2011. DOI: 10.1177/0956797611417632. Classic discussion of inflated false positives under undisclosed analysis flexibility
- [70] Gelman, Andrew and Loken, Eric. The Garden of Forking Paths: Why Multiple Comparisons Can Be a Problem, Even When There Is No “Fishing Expedition” or “p-hacking” and the Research Hypothesis Was Posited Ahead of Time. Unpublished manuscript, 2013. Accessed 2026-02 [https://sites.stat.columbia.edu/gelman/research/unpublished/p\\_hacking.pdf](https://sites.stat.columbia.edu/gelman/research/unpublished/p_hacking.pdf)
- [71] Lan, K. K. Gordon and DeMets, David L.. Discrete sequential boundaries for clinical trials. *Biometrika*, 70(3), 659–663, 1983. DOI: 10.1093/biomet/70.3.659.
- [72] Ville, Jean. Étude critique de la notion de collectif. 1939. Thèse; accessed 2026-02 [https://www.numdam.org/item/THESE\\_1939\\_\\_218\\_\\_1\\_0.pdf](https://www.numdam.org/item/THESE_1939__218__1_0.pdf)

- [73] Howard, Steven R. and Ramdas, Aaditya and McAuliffe, Jon and Sekhon, Jasjeet. Time-uniform Chernoff bounds via nonnegative supermartingales. *Probability Surveys*, 17, 257–317, 2020. DOI: 10.1214/18-PS321. <https://projecteuclid.org/journals/probability-surveys/volume-17/issue-none/Time-uniform-Chernoff-bounds-via-nonnegative-supermartingales/10.1214/18-PS321.full>
- [74] Ramdas, Aaditya and Grünwald, Peter and Vovk, Vladimir and Shafer, Glenn. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4), 2023. [arXiv:2210.01948](https://arxiv.org/abs/2210.01948).
- [75] Shafer, Glenn. Testing by betting: A strategy for statistical and scientific communication. *Journal of the Royal Statistical Society: Series A*, 184(2), 407–431, 2021. DOI: 10.1111/rssa.12647. <https://rss.onlinelibrary.wiley.com/doi/abs/10.1111/rssa.12647>
- [76] Melara, Marcela S. and Blankstein, Aaron and Bonneau, Joseph and Felten, Edward W. and Freedman, Michael J.. CONIKS: Bringing Key Transparency to End Users. 24th USENIX Security Symposium (USENIX Security 15), 2015. <https://www.usenix.org/system/files/conference/usenixsecurity15/sec15-paper-melara.pdf>
- [77] Sun, Aozhuo and Lin, Jingqiang and Wang, Wei and Liu, Zeyan and Li, Bingyu and Wen, Shushang and Wang, Qiong Xiao and Li, Fengjun. Certificate Transparency Revisited: The Public Inspections on Third-party Monitors. Network and Distributed System Security (NDSS) Symposium, 2024. DOI: 10.14722/ndss.2024.24834. Proposes watcher to inspect third-party CT monitors and detect inconsistencies <https://www.ndss-symposium.org/wp-content/uploads/2024-834-paper.pdf>
- [78] Tomescu, Alin and Bhupatiraju, Vivek and Papadopoulos, Dimitrios and Papamanthou, Charalampos and Triandopoulos, Nikos and Devadas, Srinivas. Transparency Logs via Append-Only Authenticated Dictionaries. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2019. CCS ’19; transparency logs and equivocation/monitoring tradeoffs; accessed 2026-02 <https://www.cs.yale.edu/homes/cpap/published/logs-ccs19.pdf>
- [79] Merkle Tree Certificates. IETF Internet-Draft (draft-ietf-plants-merkle-tree-certs), 2026. Internet-Draft; current versions cite C2SP tlog specs (e.g., tlog-checkpoint/tlog-mirror/tlog-tiles); accessed 2026-02 <https://datatracker.ietf.org/doc/draft-ietf-plants-merkle-tree-certs/>
- [80] I2P Technical Introduction: Network Database (floodfill / netDb). I2P Project documentation, 2026. See “Network Database” section (floodfill, store propagation, and lookup behavior); accessed 2026-02 <https://geti2p.net/ca/docs/how/tech-intro>
- [81] The Network Database (netDb) and Floodfill. I2P Project documentation, 2026. Overview of floodfill roles, routing keys, and lookup messages; accessed 2026-02 <https://i2p.net/en/docs/overview/network-database/>
- [82] An Architecture for Trustworthy and Transparent Digital Supply Chains. IETF Internet-Draft: draft-ietf-scitt-architecture, 2025. SCITT architecture for signed-statement transparency services; accessed 2026-02 <https://datatracker.ietf.org/doc/html/draft-ietf-scitt-architecture-22>
- [83] Rekor Transparency Log: Logging Overview. Sigstore documentation, 2026. Describes Rekor entries, inclusion proofs, and verification tooling; accessed 2026-02 <https://docs.sigstore.dev/logging/overview/>
- [84] Sigstore Threat Model. Sigstore documentation, 2026. Describes the Sigstore workflow and Rekor inclusion promises (SET) in the threat model; accessed 2026-02 <https://docs.sigstore.dev/about/threat-model/>
- [85] Anonymous. Notarized Anonymity Certificates: Extended Figures and Log-Hardening Notes (Addendum). Evaluation series companion note (Eval 1A), 2026. (This archive.)